POSITION PAPER

Tell Your Coding Agent to Work as an Architect First

Let it declare intent, let it show the design, let both pass the gates — then, and only then, let it code

Stanislav Rumega

21 July 2026 · rev. 5.4.1

1 The Missing Intermediate Representations

Modern coding agents are astonishing at producing code and almost incapable of saying what they are about to build. That gap — between the speed of generation and the absence of stated intent — is where the dangerous bugs now live.

Watch a coding agent work on anything non-trivial — a payment worker, a retry loop, a supervised actor tree — and a pattern emerges. It reads the issue, and within seconds it is writing code. The code is fluent, idiomatic, often correct in the happy path. But nowhere in the process does the agent state, in any checkable form, the answers to the questions that actually kill systems in production: *What happens when this crashes mid-operation? Is this command allowed to be lost silently? How many of these can exist at once? What enforces that this invariant holds when the thing enforcing it is the thing that failed? How is it stopped?* The agent may have implicit answers buried in the code it wrote. It has no explicit ones a machine — or a reviewer — can inspect.

This is not a complaint about any particular model; it is a property of the workflow. The pipeline today is `prompt → code`. Intermediate representations between requirements and code are, of course, an old idea — design docs, ADRs, RFCs on the informal side; TLA+, Alloy, UML, Design-by-Contract, and session types on the formal side. But these were all designed for a world in which *humans* write the code. The formal ones demand a rigor most teams never adopt; the informal ones are prose — unverifiable, unenforceable, written (if at all) for humans to skim, not for machines to check. What is missing is not the *idea* of an intermediate representation; it is an intermediate representation **shaped for an agent-first workflow** — one the agent itself can be required to produce, and a machine can deterministically check. The faster generation gets, the wider this gap grows. An agent that produces a thousand lines a minute with no stated intent is not an engineer; it is a very fast typist with opinions.

> **Thesis.** Before a coding agent writes code, it should have to declare — in a structured, machine-checkable form — what it believes the system is, how it can fail, and what it claims will hold. We call that artifact a **System Intent Model** (SIM), and a deterministic reviewer should validate it *before* any code exists to review. And it should have to do it twice: the approved intent is lowered into a concrete design — the **Formal Architecture Model** (FAM) — which a second deterministic reviewer validates in turn, checking the things that only become checkable once the design exists: completeness, consistency, traceability. Intent reviewed, then design reviewed, then the code itself — where mature static analyzers already exist and are adapted, not reinvented. And where the requirements themselves are too thin to declare honestly, the agent questions them — and, past the threshold, declines to build rather than guess.

The Same Thing in Plain Language

AI coding assistants start typing code immediately, without ever saying what they think they are building. So: before it writes any code, the assistant must fill in a short **questionnaire** — what parts the system has, how they talk to each other, what happens when something crashes, and what it is *not sure about*. A small, boring, reliable program — not another AI — checks those answers against a checklist of mistakes that real systems have made in production. If an answer is missing or fishy, the assistant must fix the *plan* and resubmit. Then it happens **again, one level down**: from the approved plan the assistant must draw the detailed **blueprint** — the exact tables of what each part does in every situation — and a second boring program checks the blueprint for holes: missing cases, states that can suspend unfinished work forever with no deadline, promises with no named place in the code that will keep them. Only when both checkers pass does anyone write code. And the code gets checked too — but here the industry already did the work: good automated code checkers exist for every ecosystem, so the pipeline simply *uses* them. The part nobody had built was the two checkers above it.

Why boring is the point: you do not want a second AI judging the first — it can be sweet-talked, just like the first one. The checker is a smoke detector, not a critic: simple rules, no opinions, same answer every time. *Is this message allowed to vanish silently? What stops a crash–restart loop?* If the plan does not say, that is a finding. And if *your requirements* do not say, the assistant must ask you — and if you will not answer the questions that actually matter, it must decline to guess. That refusal is not a tantrum; it is the job.

We say	We mean
SIM	the questionnaire the AI fills out before coding (a structured file)
gate / reviewer	the boring checker program — three, actually: one for the questionnaire, one for the blueprint, and off-the-shelf scanners for the code
heuristics H1–H14	the checklist for the questionnaire — “mistakes real systems have made”
checks F1–F6	the checklist for the blueprint — not stories from the past, but arithmetic: every case has a row, every promise has a place in the code, nothing appears that the questionnaire didn’t allow
blocking finding	a missing answer serious enough to stop the work until answered
FAM	the detailed blueprint drawn from the approved questionnaire — the *design*, a.k.a. the *architecture* (same thing); code must match it one-to-one
decision memory	the project diary: every design choice and *why* — so you never re-argue it next month

If the rest of this paper ever gives you a headache, this box is the whole idea. The jargon exists for precision, not for reading.

2 What a System Intent Model Is

Definition. A System Intent Model is a structured declaration of a module's intended architecture, written by the agent that is about to implement it, covering: its *actors* and their responsibilities and state persistence; its *messages* with direction, delivery semantics, and acknowledgement contracts; its *guarantees* and the concrete mechanisms claimed to provide them; its *failure model* — what can go wrong and the stated response to each;

its *resource creation* and bounds; its *enforcement mechanisms* and the layer at which each invariant is enforced; its *identity and freshness assumptions*; and, explicitly, its *ambiguities* — the things the agent does not know.

Two properties make a SIM different from the design documents engineers already write, and both are essential.

It is structured, not prose. A SIM is JSON against a schema, with enumerated fields: a message's `delivery_semantics` is one of `at-most-once / at-least-once / exactly-once / unspecified`; a failure response has a `response_kind` of `detection / enforcement / unspecified`; an invariant's enforcement has a `layer` of `resource / platform / application / none`. This is what makes it *checkable*: a deterministic program can read it and ask precise questions, because the answers are in fields, not buried in adjectives.

Absence is data. This is the discipline that makes the SIM honest. If there is no acknowledgement, the agent must write `"expects_ack": false` — not omit the field, and not write prose about how it will "handle reliability." If a failure has no response, the `response` is empty. If an invariant is enforced only by the same software it constrains, the layer is `application` or `none` — a statement that the enforcement is advisory, recorded as such. A negation ("not fenced", "no backoff") is recorded as the *absence* of a mechanism, never dressed up as its presence. In a SIM, what is *not* there is as visible as what is — a discipline closer to how a compiler treats a program than to how a design doc treats a system.

And there is a third, quieter feature: the `ambiguities` list. A coding agent that is forced to declare intent will sometimes not know the answer — is this subscriber reading a fresh value or a cached copy? what does this timeout actually do when it fires? The SIM gives it a place to say so, honestly, as a flagged ambiguity rather than a fabricated certainty. A model that can admit ignorance in a structured field is far safer than one that must guess to proceed.

3 The Right Frame: Intermediate Representations, Plural

It is tempting to read the SIM as a kind of design document that happens to be JSON — an artifact for a reviewer to look at. That undersells it. The more precise frame comes from an unexpected place: compilers.

A mature compiler does not translate source directly into machine code. It lowers the source through a series of *intermediate representations* (IRs), each one more explicit and more checkable than the last, and it runs verification passes on those IRs before any code is emitted. Note the plural — serious compilers have *several* IRs: LLVM lowers from LLVM IR through SelectionDAG to machine instructions; MLIR made “many dialects, progressive lowering” its entire design philosophy; each level has its own verifier, and the higher levels are semantic and sparse while the lower ones are structural and total. The IR is the load-bearing invention: it is what lets a compiler be *staged* — parse, then check, then optimize, then emit — with each stage trusting the last because the representation between them is precise. Nobody mistakes the IR for bureaucracy; it is the reason compilers are reliable. Our pipeline inherits the plural directly: the SIM is the top-level IR (semantic, tolerant of declared absence), the FAM is the second IR (structural, intolerant of it), the code is the emission — and each level has its own verification pass.

The SIM is exactly this, two levels up from code — with the FAM one level up, in between. Today's agents compile `Prompt → Code` in a single opaque step. The proposal here is to give that pipeline its missing IRs, one per level, as Figure 1 shows.

Figure 1. The generation pipeline with two intermediate representations and three deterministic gates. The SIM (intent level) is checked for the promises; the FAM (design level) for the arithmetic; the code (implementation level) for the defects that can be found only at the code level. The agent appears at the two IR stages, which it authors; everything to their right — the three gates — is deterministic and inspectable.

The SIM is the *first* intermediate representation — the intent-level one — between natural-language requirements and implementation, and the deterministic reviewer is the verification pass over it. The Formal Architecture Model is a *lowering* — and, by the standard of Section 3.1, itself a full IR, the design-level one: it has defined semantics, a machine-checkable form, its own verification pass (Gate 2), and a lowering beneath it (the code). This is not a terminology upgrade; it is the property that lets the FAM cross-examine the SIM (Section 6). The two IRs divide labor the way compiler IRs do: the SIM is semantic and sparse — absence and ambiguity are first-class citizens, honestly declared. The FAM is structural and total — absence is illegal there; a missing row is a finding, not a field. A note on names before we go further: throughout this paper, **“architecture” and “design” are the same thing** — the single intermediate level between the declared intent and the code, with a single set of artifacts. A companion rule, since the words invite confusion: **“requirements” name the prose the agent starts from; “intent” names what the agent declares in the SIM** — intent is requirements made explicit, structured, and checkable, not a synonym for them. The ladder is requirements → intent (SIM) → design (FAM) → code. We use “architecture” when the emphasis is on *structure* (actors, channels, supervision trees, ports) and “design” when the emphasis is on the *act and artifact of deciding* that structure; the FAM is both words at once, which is why its name can carry either. Nothing in this paper distinguishes them, and any reader who finds the two words pulling apart should report a bug in the text, not discover a distinction in the method. And the code is the final emission, traceable back through each stage. This framing does two useful things. First, it explains the architecture to any programmer in one sentence, using an analogy every programmer already trusts: *we are not inventing bureaucracy; we are moving software generation toward the same staged, verified pipeline that made compilers reliable.* Second, it makes the downstream pieces fall out naturally — the gate, the FAM, the compliance report, and even the code are all just *consumers* of the IR. Get the IR right, and the rest of the pipeline is ordinary engineering. Get it wrong, and no amount of downstream rigor helps — which is precisely why the next two sections care so much about reviewing it, and about whether the agent can write it accurately.

There is a deeper reason the IR framing earns its keep: **a useful IR is the contract between stages.** In a compiler, each IR is the fixed point the passes on either side agree on — produced faithfully, preserved semantically, reasoned about directly. The same holds here, at both levels. Gate 1 reasons *about the SIM*; the FAM *refines* it without changing its meaning; Gate 2 reasons *about the FAM*; the code *implements* it; and traceability is the demonstration that the semantics were preserved across each stage. Once the IRs are understood as the contracts, the pipeline's stages stop being a sequence of ad-hoc artifacts and become a single, composable whole — which is exactly the property that made compiler IRs so powerful.

3.1 Properties of the IR

Promoting the SIM to IR status invites the questions compiler researchers ask of any IR. They deserve direct answers.

- **Is it lossless?** No — and deliberately so. An IR is an abstraction, not a transcription. The SIM records the failure-relevant semantics of the design, not every implementation detail. The right property is not losslessness but *adequacy*: does it capture everything the verification passes need to reason about? That is an empirical question, answered by the rubric and the cold-test, not assumed.
- **Is it canonical?** No. Many prompts can normalize to the same SIM, and the same intent admits multiple valid SIMs. Canonicity is not the goal; *fidelity* is — the SIM must describe the system the agent actually intends to build, which is precisely the "code match" property the validation measures.
- **Is it deterministic?** Split, by design. *Generation* of the SIM is stochastic (the agent proposes). *Review* of the SIM is deterministic (the same SIM always yields the same verdict). This is the central asymmetry of the whole architecture: a stochastic front end, a deterministic verifier.
- **What is the transformation invariant?** Not injectivity — *traceability*. Every FAM element cites the SIM element it refines; every FAM invariant names the code location that will enforce it. The invariant across stages is that meaning is preserved and *provably so*, by an audit trail, rather than by a one-to-one mapping. Enforcing that the code still honors the FAM across the *next fifty edits* is the declared-vs-actual drift problem — the pipeline's hardest open stage, addressed in Section 7.1.

Answering these plainly is not a concession; it is what separates an IR from a data format. A JSON blob becomes an intermediate representation when its properties — what it preserves, what it abstracts, and what the stages may assume about it — are stated and defended. The FAM answers the same questions at its own level; that is what makes it the second IR rather than a rendering of the first.

It is worth adding that this pipeline is an old idea whose tooling has only now caught up. In the extreme case the layering collapses entirely: an actor whose behavior is a complete Mealy table stored as data and walked by stable plumbing has its design and its code *coincident* — one artifact, two roles — and the LabHSM toolkit, an early actor-model framework for LabVIEW whose actors were defined by hierarchical state machines, said so on an animated banner twenty years ago: *“Design is code!”* The claim was right then; what was missing was a machine that could check the design before running it. That is the piece this paper adds.

4 Why Review Is Necessary — and Why It Must Come First

The case for reviewing the SIM before code rests on three observations, each drawn from real failures rather than theory. As will become clear, each observation applies again one level down — to the FAM before code — because the FAM is where the intent becomes concrete enough to check completely; we note the echo where it lands.

4.1 The Bugs That Matter Are Design Bugs, and They Are Invisible in Code Until They Fire

The incidents that take down production systems are rarely syntax errors or off-by-one mistakes. They are failures of intent: an acknowledgement that was never required, an instance count with no bound, an identity tied to a reusable label, an invariant enforced by the very party it was meant to constrain. We distilled a set of review heuristics from fifteen real systems — actor clusters, Kafka ingestion pipelines, consensus implementations, cellular architectures,

durable-execution runtimes, an industrial control system — and the pattern is consistent. A subset of these bugs is invisible even at the intent level and only becomes *checkable* at the design level: a missing (state, event) case is not a promise anyone failed to make — it is arithmetic that does not yet exist until the table is written. That is why the review runs twice: the SIM review catches broken and missing *claims*; the FAM review catches incomplete *constructions*. The Petabridge split-brain was a node-identity assumption (a reusable `hostname:port` mistaken for a unique incarnation) plus a stale local view of cluster membership; both were design decisions, both were invisible in any single diff, and both were catastrophic. A PagerDuty outage was an unbounded producer created per request against a shared broker — a resource-bound bug, present in the design before a line of code was written. These are not coding errors. They are *declared-intent* errors, and they can only be caught where intent is declared.

4.2 Reviewing Intent Is Cheaper and Earlier Than Reviewing Code

A code review of a thousand generated lines is slow, and it anchors the reviewer on what *is* rather than what *should be*. A SIM is a few hundred lines of structured declaration that can be reviewed deterministically in milliseconds — and, crucially, *before* the code exists to anchor anyone. Catching "this command is fire-and-forget with no acknowledgement" at the SIM stage is a one-line fix to a design decision. Catching it after the code is written means unravelling an implementation. The economics are the same argument the industry already accepts for shifting testing and security left — applied to intent, which is further left than either. The FAM sits one rung closer to code but still far above it: a complete Mealy table for a payment worker is a page of rows, checked in milliseconds, where the equivalent assurance from code means reconstructing the table from control flow — the extraction problem we deliberately avoid by declaring first. Each level down costs more to check than the level above it and less than the level below; that gradient, not dogma, is why the gates are ordered the way they are.

4.3 An Agent That Must Declare Intent Catches Its Own Mistakes

There is a subtler benefit, and it may be the largest. The act of writing the SIM forces the agent to confront the design before committing to it. An agent that has to write down `"delivery_semantics": "at-most-once", "expects_ack": false` next to a field it has named `ChargePayment` has, in that moment, a chance to notice the problem itself — before any reviewer runs. Declaring intent is not only an input to review; it is a discipline that improves the design being declared. In our closed-loop reference run, the agent's second and third SIM proposals were not merely compliant — they were *better designs*: the fire-and-forget alarm gained an acknowledgement, the in-memory dedup moved to a resource-layer constraint, the unbounded instance pool gained a bound. The gate's findings drove the improvement, but the declaration is what made the improvement possible. The same discipline operates at the FAM level, in a sharper form: an agent that has to write the *row* for “ShutdownWorker arrives while Charging” cannot leave the case unconsidered — the table format itself forces the question. Half of what Gate 2 catches is not disagreement with the agent's design but the design's unexamined corners, made examinable by the requirement to fill them in. To be clear about the epistemic status: this is a single anecdote — an *existence proof* that the loop can run and can improve a design, not evidence that it does so reliably. Reliability is exactly what the blind A/B cold-test (Section 6) is designed to measure, and nothing in this paper should be read as pre-empting that experiment.

4.4 Loop Zero: Question the Requirements — and the Right to Refuse the Build

There is a loop before the first gate, and it is the only one whose converging partner is not a machine. An agent told to work as an architect will sometimes — often — discover that the prose requirements it received are not enough to fill the questionnaire honestly: no word on what happens when the gateway is down, nothing about whether the

command may be duplicated, silence on who may stop the system. Today's agents paper over such gaps with plausible defaults and type on. That is precisely the behavior this pipeline exists to stop — so the discovery itself must be part of the method.

Loop Zero works like this. The agent drafts what it can, and for each gap it cannot honestly fill it records a *blocking ambiguity* — the SIM schema has a place for exactly this, severity and all. Blocking ambiguities do not route to another guess; they route *back to the human*, as questions posed in the SIM's own vocabulary: not “tell me more about the system,” but “when the payment gateway is unreachable mid-charge, should the worker retry with a bounded backoff, park the order, or escalate?” Each question is a schema slot, each answer edits fields rather than prose, and the exchange is the decision memory's first and often richest feed — for many systems, this interrogation is the first time the requirements were ever written down in checkable form. Two disciplines keep the loop survivable: questions are *batched*, and each carries the agent's default proposal (“if unanswered, I will assume at-least-once with an idempotent receiver”) — defaults make silence answerable, the record makes it honest.

None of this abolishes the ambiguities list — it *routes* it, by severity, and the routing is the design. **Blocking** ambiguities interrogate, and block the build if unanswered. **High** ones are asked, but the agent may proceed on an explicit default — on record, as always. **Medium and low** ones simply stay in the document: honestly named, never silently guessed. A SIM with a populated ambiguities section is healthier than one pretending completeness, because the list is where the honesty lives. The discipline is asymmetric on purpose: the agent must *surface* everything — exhaustively — but may only *refuse* on the things where the failure modes live. Surface everything, refuse selectively: a pipeline that interrogates the user about trivia is uninstalled within a week, exactly like a gate that cries wolf. And one refinement worth stating: if the user answers a blocking question with “I don't care, just pick,” that is a *resolution*, not a refusal — the human delegated, the decision memory records the delegation (“user accepted agent default X”), and the build proceeds with the default marked user-sanctioned. Even indifference becomes a decision, on the record.

One more refinement is forced by reality: **the user is often not qualified to answer, and that has to be survivable.** A bakery owner who wants a loyalty app cannot say what delivery semantics her payments should have — and should never be asked in those words. The first move is therefore *translation, not omission*: the agent poses each blocking question in consequences the user owns — “if a customer pays and the connection drops, is it worse to charge them twice or to lose the order?” — and maps the answer back to the schema slot. The SIM vocabulary is for the gate; the user speaks outcomes, and a person who cannot define idempotency can still say “worse to double-charge.”

And where even a translated question cannot be answered, the second move: **the agent is allowed — expected — to know better than the user.** This is the part of the design that inverts the usual flow of authority, and it should be stated plainly. The user owns the *what* and the *why*: what the system does, what it may never do, what a failure costs in their world. The agent routinely knows the *how* better than any individual user can: which delivery semantics make double-charging impossible, which dependency versions carry known vulnerabilities, which logging satisfies an auditor — and what the law now requires of software in its class. Regulatory constraints like the EU Cyber Resilience Act are the sharpest case: the user has never heard of an SBOM, yet the agent applies the CRA's secure-by-default checks at the SIM, the FAM, and the code, and emits the compliance report and the bill of materials as byproducts — the user discovers their obligations existed only when they are handed the evidence of having met them. These choices go on record as *posture defaults* (“strictest reasonable option assumed, user did not override”), and the gate may *require* the strict default where the SIM declares a compliance scope. The asymmetry is deliberate and defensible: what the user must answer is what only they can know — the system's purpose and its irreversibles; what the agent must supply is what only it can know — the state of the art and the state of the law. A harness that

waits for users to become experts protects nobody; a harness that applies expertise and records the application is precisely the point of having one.

And if the human will not cooperate past the blocking threshold — that is, cannot or will not answer even the purpose-level questions, the ones about what the system should *do* and what it must never do — then the agent must be allowed — required — to **decline the build**. This is not petulance; it is the same professional standard every mature discipline already enforces: a structural engineer does not sign off on vague loads, a surgeon does not operate on "somewhere around here." An agent that proceeds on unresolvable ambiguity is not being helpful, it is being negligent — it will produce a SIM that passes every gate while describing a system the user never asked for, a compliance certificate for a fantasy. Note who does the refusing: not the agent's judgment, but the pipeline's — a blocking ambiguity means Gate 1 cannot pass, which means no FAM, which means no code. The refusal is reproducible, auditable, and immune to sweet-talking, which is exactly why it lives in a deterministic gate rather than in the agent's discretion. And it is a *report*, not a sulk: the deliverable of a refused build is the documented hole — the blocking ambiguities, the questions, the defaults proposed, the options considered — often the most valuable artifact the project has produced so far. One legitimate exit remains: *shrink the scope* to what is genuinely specified ("the logging component's requirements are complete; the payment path stays unbuilt"). Building the well-specified subset is cooperation; widening the guessed scope to look complete is not. Trivial gaps never trigger any of this — that is what defaults-with-record are for. Refusal is reserved for ambiguities where the failure modes live: delivery semantics of money-moving commands, shutdown authority, identity assumptions. The right to refuse is what makes the agent an architect rather than a typist. Architects can be wrong on the record; typists just retype.

5 Why the Review Must Be Deterministic — Not Another Model

If the SIM — and the FAM derived from it — is to be reviewed, the obvious question is: reviewed by what? The tempting answer — another LLM acting as a judge — is precisely the wrong one, and it is worth being clear about why.

A stochastic judge cannot be the fixed rulebook. The entire point of the review is to be a hard, reproducible constraint on a fast, probabilistic generator. An LLM reviewer is itself probabilistic: the same SIM may pass today and fail tomorrow, and the agent being gated can learn to write SIMs that *please the judge* rather than SIMs that are *sound*. This is the LLM-judges-LLM trap, and it recreates the original problem one level up. Worse, an LLM judge's reasoning is not auditable — you cannot open it and read the exact rule that fired. The review we need must be the thing the generator is *not*: slow, small, deterministic, and inspectable.

The principle, stated plainly: the stochastic mind proposes; the deterministic record disposes. The reviewer is a fixed set of heuristics — thirteen of them, each traceable to a real incident or a named failure class — implemented as ordinary, auditable code. The same SIM always yields the same report. A design partner, or a regulator, can read the rule that produced a finding. That determinism is not a limitation; it is the entire value. It is also what makes the review *composable* into a gate: a deterministic check can block a build; a probabilistic opinion cannot.

6 The Feasibility Question: Can an Agent Even Write a Good SIM?

This is the honest crux, and we will not pretend it is settled. The entire pipeline stands or falls on a single assumption: **that the SIM captures the agent's actual intent accurately.** If the agent writes a tidy, internally-consistent SIM that describes a system other than the one it is about to build, then the deterministic review, the

formal model, and the code can all be perfectly consistent with one another — while implementing the wrong thing. This is the risk an independent reviewer named when shown the architecture, and it is the correct diagnosis.

Three things make the assumption *plausible* rather than naive — and a fourth belongs to the FAM: the lowering is a *second, independent sampling* of the same intent. An agent whose SIM quietly described the wrong system would have to carry the same misreading, consistently, through a far more constrained artifact — typed messages, complete tables, named code locations — and then past a gate that checks the FAM against the SIM mechanically. Drift between SIM and FAM is detectable in a way drift between SIM and code is not, precisely because both artifacts are structured. The FAM is thus not only a design step; it is the intent's first cross-examination.

Declaration is easier than extraction. The hard version of this problem — reading existing prose or code and reconstructing its intent — is genuinely difficult and error-prone. But that is not what we are asking. We are asking the agent to *declare* its intent before implementing, in a structured form it is entirely capable of emitting. Producing JSON to a schema is squarely within a coding agent's competence; the discipline is in requiring it, and in refusing to proceed without it. The SIM inverts the extraction problem: you do not recover intent from artifacts, you require intent as a precondition for artifacts.

The schema is honest about ignorance. The anti-fabrication disciplines — absence is data, negation is absence, ambiguities are allowed — mean the agent is never forced to invent a mechanism to proceed. When it genuinely does not know, it says so in `ambiguities[]`, and the gate surfaces that as a question for a human rather than letting it harden into a false claim. A declaration format that has a first-class place for "I don't know" is far more likely to be accurate than one that must always produce an answer.

It is measurable, and we intend to measure it. The assumption is not to be taken on faith; it is to be tested. The validation protocol is a blind A/B comparison: the same agent solves the same issues with and without the gate, and the SIM is scored on a rubric whose most diagnostic dimension is *code match* — does the declared SIM describe the code the agent actually wrote? If SIMs are routinely tidy but describe a different system, the assumption is falsified and the declaration contract must be fixed before anything downstream is trusted. If they track the code, the gate is reviewing the real design. This is the experiment that decides whether the whole approach is sound — and it is designed to be able to fail.

7 The Method: The Loop That Enforces It

Assembled, the method is **three closed loops with three deterministic gates** — one per level of commitment. The agent declares intent; Gate 1 checks the promises; the loop repeats until they hold. The approved intent is lowered into a design; Gate 2 checks the arithmetic; the loop repeats until it is complete and consistent. The approved design is emitted as code; Gate 3 — the static analyzers every ecosystem already has, adapted and routed — checks the emission; the loop repeats until it is clean. Three times the agent proposes, three times a deterministic pass disposes, and only the third proposal is the one anyone wanted in the first place.

```
prose spec
  → LOOP 0: blocking ambiguities → questions to the human
        ↑___ answers edit SIM fields (defaults on record) ___↓
        (no cooperation past the threshold → decline, deliver the evidence)
  → agent DECLARES a SIM
  → GATE 1 reviews it (H1–H14, intent)
        ↑________ blocking findings fed back ________↓
  → iterate until the SIM passes
  → compliance report + SBOM (byproducts of the passing review)
  → agent DERIVES the FAM — the design, machine-readable
  → GATE 2 reviews it (F1–F6, design)
        ↑________ blocking findings fed back ________↓
  → iterate until the FAM passes
  → code, mirroring the FAM one-to-one
  → GATE 3 reviews the code (existing SAST tools, adapted)
        ↑________ blocking findings fed back ________↓
  → iterate until the code passes
  → every decision logged to the project's decision memory
```

Each stage exists for a reason, and each is only trustworthy because the one before it was gated.

Gate 1 runs thirteen heuristics — failure-mode completeness, delivery coherence, bounded recovery, no unbounded waits, identity and freshness integrity, verification strength, independent enforcement of invariants, bounded resource creation, operation reversibility, invariant conflicts, conditional guarantees, and corrigibility (can the system be stopped on command: enforced, inherited, and not through a single point of failure) — plus three property-based additions covering actors whose behavior cannot be enumerated, and a supply-chain check on every external dependency. **Gate 2** runs six of its own over the FAM (F1–F6, Section 7.4): table completeness, block–property match, lowering integrity, port coherence, guard-as-code, code-mirror readiness. **Gate 3** adapts existing analyzers for the code (Section 7.5). The full question lists are in the Appendix; the complete implementations, with severities and applicability conditions, are a few hundred lines of ordinary, auditable code each. Findings are severity-ordered, each with the evidence and the question to ask. A *blocking* finding stops its loop; a strength is confirmed as sound engineering to keep.

The iteration is the point. When a gate blocks — at any of the three levels — the findings go back to the agent — not as a human's red ink, but as the precise, structured reasons the declared design is unsound. The agent revises the *design* (not hides the gap) and resubmits. In our reference run, the first SIM was blocked (fire-and-forget payment and alarm commands, no failure model, a guarantee with no mechanism), the second was blocked again (an in-memory "enforcement" that was really advisory, an unbounded instance pool), and the third passed — with a resource-layer idempotency fence, a bounded pool, a bounded restart policy, and a declared shutdown path. Three rounds, each one a measurably better design.

The FAM and the code come last, and only then. Once the SIM passes, the agent derives the design — the Formal Architecture Model, i.e. the architecture, written down: typed messages, explicit state machines with per-state transitions, named invariants mapped to their enforcement layers and code locations — and Gate 2 reviews that design with its own loop (Section 7.4) before any code is written. Only when both gates have passed does the agent write code that mirrors the FAM one-to-one — and Gate 3 then reviews the code itself (Section 7.5). The code is no longer the first artifact; it is the last, and it is traceable back through the FAM to a SIM that a deterministic gate accepted. This is the agent-native realization of an old ambition — the unambiguous specification that precedes the

program — made practical because the specification is now something the agent itself must produce and a machine can check.

And every decision is remembered. Each check, block, and override — especially the human overrides with their stated rationale — is appended to the project's decision memory, an append-only log that outlives any single session. (One could call it the project's “constitution”; we will resist that temptation.) Its one non-negotiable clause is that the agent may not amend it unilaterally; amendments are human decisions, recorded. This is what turns a static gate into a learning system: the heuristics themselves were each distilled from a real incident, and the memory is where the next ones come from.

7.1 Keeping the Declaration True: The Drift Problem

The sharpest objection to everything so far is not about the first build — it is about the *next fifty edits*. A SIM approved on Monday is a claim about a system that, by Friday, has absorbed a hotfix, a new message, and an “obviously harmless” timeout change. If the declaration silently stops describing the code, the gate becomes a ceremony: impressive on day one, decorative by week two. This is the drift problem, and a position paper should say plainly how the architecture intends to meet it.

The design answer is that **drift is checkable because extraction is symmetric to declaration**. The same pipeline that lets the agent *declare* a SIM before code can be run in reverse over existing code — re-extracting a candidate SIM (or FAM) of what the code *actually does*. Drift detection is then not a semantic guessing game but a **diff between two structured documents**: the approved SIM and the re-extracted one. New message without a declared delivery semantics? A failure response quietly emptied? An unbounded pool where the SIM declared a bound? Each of those is a diff line, and the gate can run on the diff exactly as it runs on a fresh SIM — blocking, with a question, when the drift crosses a rule.

And drift is not always sin. Much of it is the system *learning*: the hotfix that adds a bound is a design improvement discovered in production. So drift is routed through the same discipline as everything else: the human reviews the diff, and if the new reality is accepted, the SIM is amended *with a recorded rationale* — a decision event in the memory, which is precisely the feed that grows the knowledgebase (Section 8). Drift the human rejects is a bug to fix; drift the human accepts is a lesson to keep. Either way the declaration, the code, and the record stay mutually honest.

Honesty about status: the extractor exists and has been run against real frameworks (Section 8.1's industrial actor framework was machine-extracted), but the re-extract–diff–gate loop is *prototype-stage*, not yet a daily tool. The drift check is also what makes the FAM earn its keep: FAM elements name the code locations that implement them, which is what gives the diff somewhere to bite. We state the design here because it follows from the architecture — not because we claim it is finished. If the cold-test says the SIM is accurate at birth, keeping it accurate is the experiment after that.

7.2 The FAM Without UML: One Skeleton, Four Evidence Shapes

A fair skepticism meets any proposal for a “formal architecture model” in 2026: *UML is dead, and good riddance — what makes your blueprint notation any different?* UML died of three causes: too many diagram types, no executable semantics, and tooling that could not keep picture and code in sync. The FAM avoids all three by not being a notation at all. It is a fixed skeleton with a small set of *evidence shapes*, and which shape an actor uses is determined by the same declared properties the gate already reads — never by a family label.

The skeleton is always the same five parts: **ports** (each with domain, dimensions, rate, bounds), **state** (its dimensions and persistence), **behavior** (one block, below), **guard** (the deterministic envelope around the behavior, as code with a named location), and **traceability** (which SIM elements this refines; which code locations implement each part). The behavior block comes in exactly four shapes:

Block	When (property, not label)	What the agent writes
T — Table	enumerable + reproducible (FSM, HFSM, behavior tree)	The full Mealy table, (state, event) → (actions, next state), every row including unexpected-event rows; hierarchies resolved to their leaf-level table. This is the case where the table *is* the code's behavior-data — the FAM and implementation coincide, and the gate can compute over the rows directly.
E — Envelope	not enumerable or not reproducible (learned policy, stochastic, genetic/swarm)	What cannot be listed must be enclosed: the valid input domain (stated symbolically), the guardrail predicates every output must satisfy (each with its code location), the deterministic fallback, and a few sampled *witness* examples for human review — spotlights, not the map.
D — Dynamics	continuous/hybrid ports (control loops, signal chains)	The update law stated symbolically (or as a named code location), per-port range, saturation and rate, and the invariants over them (anti-windup, slew limits). May combine with T or E.
G — Generative	code-generating machinery (parsers, table compilers)	The specification the generator consumes, the determinism guarantee (same spec → same artifact), and the validation step that checks the generated artifact.

7.3 Yes, This Is the Design — and No, There Are No Diagrams

At this point a certain kind of reader — let us call him, charitably, *the experienced architect* — shifts in his chair. He has been patiently translating our vocabulary into his own: the SIM is “requirements,” the gate is “review,” the FAM is “design” (or, in his other dialect, “the architecture” — same level, same artifacts, as Section 3 established). And now the translation produces an alarm: if the FAM is the design, *where are the diagrams?* No class hierarchy lovingly arranged, no component boxes with little gears, no sequence arrows choreographed across a swim-lane. Just tables, envelopes, port contracts, and traceability citations. To him, this does not look like design. It looks like design *skipped* — a pipeline that lurches from requirements straight into implementation, the way juniors do.

He deserves a real answer, because his mapping is correct. The FAM *is* the design — it occupies exactly the slot UML diagrams occupied in the process he grew up with: the moment where structure is decided and recorded before implementation begins, the artifact a newcomer is handed to understand the system, the thing review meetings are about. What has changed is not the slot but the *shape of the artifact that fills it* — and the change is forced by two facts about who now writes and reads it.

The primary reader is no longer a person. A UML diagram was a human-to-human compression format: a senior engineer could gesture at boxes in a meeting, and the semantics lived in the viewers' heads. That is precisely why it rotted — nothing in the picture could disagree with the code, so nothing stopped it from doing so. The design artifacts in this pipeline are read first by machines: the gate checks them, the drift differ compares them against the code, the compliance report is generated from them. A design whose reader is a machine must be *data* — structured, typed, absence-explicit. A picture cannot be diffed; a table row can. The experienced architect is right that something is lost when the whiteboard closes; what is gained is that the design can now be *false* in a machine-detectable way, which is the property that makes artifacts trustworthy at all.

The primary author is no longer reliable. A human designer draws what he intends and the drawing is the commitment; a coding agent produces whatever satisfies its local objective, and the only design worth having from it is one a deterministic checker can hold it to. A design artifact whose writer is stochastic must be *checkable* — every claim in it must be the kind of claim that can be verified mechanically, against a schema today and against the code tomorrow. Tables and envelopes are; swim-lanes are not. This is the deeper reason the FAM looks the way it does: it is design written in the only register that survives contact with an author who is fluent, fast, and not accountable.

None of this confiscates the diagrams. People genuinely need the at-a-glance overview — the supervision tree, the message choreography, the statechart — and they can have it, on one condition: **the picture is a view, not the source.** Every diagram in this pipeline is *generated* from the FAM the way API documentation is generated from code: always current, never authoritative, disposable without loss. The moment a diagram becomes editable it becomes a second source of truth, and two sources of truth are zero sources of truth — that, not aesthetics, is what actually killed UML tooling. The experienced architect keeps his wall chart; he just stops being able to "fix the design" by dragging a box. He will find, after the initial sting, that fixing the table and regenerating the picture takes less time than the argument the picture used to start.

So: yes, the FAM is the design. It is design after the audience changed — written for the reader that can actually check it, with diagrams regenerated on demand for the readers who cannot. The discipline was never the pictures; the discipline was deciding, explicitly, before building. That discipline is still here. It just types now.

Two properties of this design matter more than the blocks themselves. First, **the blocks are evidence shapes, not notations**: T is evidence by enumeration, E by enclosure, D by bounds, G by determinism-of-generation. A behavior paradigm nobody has seen yet — neuro-symbolic policies, diffusion planners — needs no new block; it lands in E, because E is defined by a property of the behavior, not by a school of thought. Second, **the gate knows which block to expect**: it reads the actor's declared properties from the SIM and checks that the FAM carries the matching shape — so the agent cannot dress a learned policy in a table, or hide a missing envelope behind a notation that cannot express one. Where pictures are wanted, they are *generated views* of these blocks (a statechart rendering of a table, a message-sequence rendering of the ports); where proofs are wanted, the blocks lower to verification targets (a table to a TLA+ next-state relation). The source of truth stays textual, diff-able, and checkable — which is precisely the trio of properties whose absence killed UML.

7.4 The Second Gate: Checking the Arithmetic

Everything through Section 7.3 solves the problem of getting the design *written*. It does not solve the problem of getting it *checked* — and a pipeline that gates intent but takes the blueprint on faith has merely moved the trust bottleneck one stage down. The first version of this pipeline had exactly that hole: the gate looped at the SIM level, and the FAM was derived afterward, unchecked. That was a mistake, and it is worth being public about its shape because the shape is instructive: the checks that *can* be run at the two levels are different in kind.

The SIM gate checks **promises**: is there a response to every applicable failure mode, a mechanism for every guarantee, a bound on every resource? These are questions about what the design *claims*. The FAM gate checks **arithmetic**: does the Mealy table actually have a row for every (state, event) pair? Does every waiting state actually carry a timeout? Do the blueprint's ports match the interface the SIM declared? Does every guardrail name the code location that will enforce it? These are not questions to ask an author; they are facts to compute over a data structure — and they can only be computed once the design exists as data, which is exactly what the FAM is (Section 7.2). The loop therefore exists twice, once per level: *declare the SIM, gate it, iterate; derive the FAM, gate it, iterate; only then write code.* The second gate runs a separate family of heuristics (F1–F6: table completeness, block–property

match, traceability and no-invention, port/bounds coherence, guard-as-code, code-mirror readiness), implemented, like the first, as a few hundred lines of deterministic code.

Two symmetries fall out, and both were designed rather than found. First, the intent gate gets *lighter* in compensation: checks that were approximate at the SIM level because the information was not yet there (does this state machine have timeouts?) move down to the FAM gate, where they are exact — the SIM keeps the contract ("behavior is enumerable; unexpected events are defined"), the FAM gate verifies the rows. Second, the drift story (Section 7.1) now mirrors the build story: re-extract a candidate SIM and diff against the approved one; re-read the behavior tables and diff against the FAM. Declare → check → refine → *check again* → emit → re-extract → diff → check forever. The experienced architect from Section 7.3 may note that his old process had design review too; it did — a meeting. Ours is a program. And even two gates leave the last hole: code that faithfully mirrors an approved design can still be *bad code* — which is what the next section closes.

7.5 The Third Gate: Not Reinventing the Wheel, Routing It

The pipeline's final hole is the most obvious one: code that faithfully mirrors an approved design can still be *bad code* — an OS-command injection in a handler, a pickle load on untrusted bytes, a `while True` with no exit, a bare `except` swallowing the shutdown signal. Neither Gate 1 nor Gate 2 can see these; they live below the design, in the emission. Static analysis at this level is a solved problem, and has been for years: every serious ecosystem has mature SAST tooling — and the ecosystems closest to this project's heart are no exception, with LabVIEW alone fielding VI Analyzer for standards and bug classes, and JKI's J-Crawler (in the JKI Security Suite) scanning entire repositories for vulnerabilities, deadlocks, and memory issues, wired into CI/CD, and aimed at exactly the compliance-driven industries (aerospace, defense, automotive) this pipeline cares about. It is worth pausing on the asymmetry this reveals: **code-level checking is rich, mature, and automated; architecture-level and intent-level checking — even for human authors — is a meeting.** The industry built excellent microscopes for the smallest level and left the two levels above it to vibes and slide decks. Gates 1 and 2 exist because nothing else did; Gate 3 exists because plenty does — and the correct move is therefore not to build it, but to *adapt* it.

So Gate 3 is a **universal adapter over existing analyzers**, not a new scanner. Three design rules keep it in the spirit of the other two gates. First, *findings are routed by architectural role, not dumped by tool*: a shell-injection finding inside a handler the FAM named for a declared message is more dangerous than the same finding in generated glue — so the adapter maps findings onto the FAM's traceability (code locations again earning their keep) and reports accordingly. Second, *severity budgets stay independent per gate*: Gate 3 blocks on high-confidence security or correctness findings in declared safety/enforcement paths, and degrades gracefully elsewhere — the false-alarm economy of Section 9 applies at every level separately. Third, *language neutrality*: VI Analyzer and J-Crawler for LabVIEW, Bandit/Semgrep/ESLint for text ecosystems, `go vet`/Clippy where applicable — normalized into one findings schema, with the tool versions recorded in the decision memory, because a gate whose own configuration isn't reproducible has learned nothing from its neighbors. And the loop closes upward: a code-level finding class that keeps recurring is precisely the evidence that promotes a new heuristic into Gates 1 or 2 — the code review teaching the design review what to ask next.

8 The Knowledgebase Grows — and So Does the Schema

Everything said so far could leave the impression that the reviewer is a fixed checklist — thirteen heuristics, frozen at build time. That would be a misreading, and an important one to correct, because the component that compounds is not the gate or the schema but the **knowledgebase** behind them. The reviewer is a learning system with a

deterministic core: the heuristics are not a closed set but the current state of an accumulating record, and the architecture is designed so that record grows with every failure it digests.

The growth mechanism matters more than the current contents. Each heuristic is *born from a decision event*: a documented failure, the correction it forced, the rationale, and the rule that emerged — recorded with full provenance. Adding a new failure class is therefore not "edit the code"; it is "record the event, and let the heuristic emerge from it." The present taxonomy — thirteen heuristics across fourteen failure classes — was distilled this way from fifteen real systems, each rule traceable to the incident that produced it. The knowledgebase is the asset; the heuristics are its current projection.

Here the honest constraint enters, and it is the sharpest limitation of the whole approach. **Public architecture–postmortem *pairs* are scarce.** Incident write-ups are plentiful, but a postmortem tells you what broke after the fact; it rarely ships with the original architecture document a SIM would have been declared from. The method needs *declared-intent* ↔ *occurred-failure* pairs to learn "this intent leads to that failure," and the public record mostly supplies the failure without the declared intent. Building the seed taxonomy required reconstructing intent from postmortem prose — workable for deriving heuristics by hand, but exactly the circularity the live validation is meant to escape. So the present knowledgebase is best understood as a *curated seed taxonomy with a growth engine*, not a model trained on an abundant corpus.

And that scarcity is, in a sense, the point — because it is what justifies the growth loop. The knowledgebase does not depend on the limited public literature forever. Its primary growth channel is *live use*: every time a human overrides a gate finding on a real project, marking it right or wrong with a stated rationale, that becomes a decision event. The decision memory the pipeline writes is precisely this feed. The public pairs bootstrap the taxonomy; deployment is what feeds it. Were design–failure pairs abundant, one could batch-train and skip the loop; because they are scarce, the loop — already built — is the load-bearing growth channel. The cold-test is what starts feeding it live data.

8.1 Does the SIM Schema Itself Evolve?

A natural follow-on: if new *fundamental* constraints surface from future failures, does the schema change to make them expressible? Yes — but slowly, and by design. The distinction to hold is between the *knowledgebase*, which grows continuously, and the *schema*, which is the contract and must stay stable.

Most new failure knowledge does *not* require a schema change. A new failure class becomes a new decision event and, if it generalizes, a new heuristic — H14, H15 — expressed over the existing fields. The taxonomy has already grown this way: the corrigibility check (H13) was imported from an external framework without touching the schema, because "can the system be stopped on command" is expressible through shutdown messages, enforcement layers, and supervision edges the schema already had.

A schema change is warranted only when a new constraint is *inexpressible* in the current fields — when the failure class needs a dimension the schema has no field for. That has happened, and the pattern is instructive. Meeting a machine-extracted industrial actor framework exposed four gaps the prose-oriented schema had never needed: actor *inheritance* and effective-API resolution, *message kind* (command/query/event/self), *channel kind* (supervision-tree vs. cross-tree handle — the structural escape hatch that keeps recurring in agent runtimes), and *extraction provenance* (was this SIM code-scanned or LLM-inferred — a trust level on the IR itself). These are queued as candidates for the next schema version, precisely because they are structural and inexpressible today.

The harder test of that rule came from a different direction: variety in the *kind* of system being declared. The first schema silently assumed the actor mainstream — deterministic state machines exchanging asynchronous messages — and each challenge from outside that mainstream forced the same three-step escape. When an actor's inference might be a hierarchical state machine, a behavior tree, or a neural network rather than a flat FSM, the first impulse

was an enum of representations. When determinism itself turned out to be an independent axis (a stochastic FSM exists; a learned policy can be run deterministically), the enum grew. When genetic and swarm algorithms arrived, and then continuous-valued, multi-dimensional, multi-input/multi-output behaviors, the enumeration impulse collapsed under its own weight: every new family would need a new enum value and a new special-case rule — whack-a-mole dressed as schema evolution.

The resolution, and the design lesson this paper most wants to transmit: **test the property, not the label.** The schema stopped asking *what kind of thing* an actor is and started asking two booleans about its behavior — *enumerable* (can the (state, inputs) → behavior relation be listed?) and *reproducible* (same inputs, same behavior?) — plus structural facts: ports with domains (discrete/continuous/hybrid) and dimensions, and multidimensional state. The verification standard is then *derived*: enumerable + reproducible actors get enumeration and code-matching; anything else must declare a deterministic *envelope* (input domain, guardrails, fallback) — the stochastic part proposes, the guard disposes (H5e); continuous or stochastic outputs need bounded ranges (H5f). Learned policies, samplers, bandits, genetic and evolutionary search all fall into those property cells *with no schema change per family*. The same discipline replaced the actor-only failure model: which failures are even applicable — message loss, restart loops — is now conditioned on the declared architecture shape (H1 asks a synchronous system nothing about lost messages, because a synchronous system cannot lose one).

Two boundary decisions complete the picture, and both are the same kind of decision. **Mailboxes:** real actors have multiple input mailboxes and output channels, and the choice between a single logical priority queue (physically: per-priority FIFOs plus a wake-up queue) and multiple physical mailboxes is a genuine trade-off — global ordering simplicity versus isolation and independent backpressure. The schema therefore records the structure *and the rationale for the choice*, and the gate checks the failure modes of whichever structure was chosen — starvation across mailboxes, lowest-priority dequeue latency in a priority queue — without prescribing the choice (H5g). Neutral on the decision, strict on its defense. **Petri nets:** genuinely out of scope — a Petri net's *marking* is not an actor state, and its structural invariants are a different question family, better served by lowering a SIM *to* a net (or to TLA+) as a verification target than by stretching the SIM to be one. An IR, like a schema, earns trust by what it declines to absorb.

So the rule is: **heuristics grow freely; the schema versions deliberately.** The schema is versioned, and because it is the IR, a version bump is a contract change — every consumer (gate, FAM, transpilers) must be able to read old and new SIMs. Criteria for promoting a candidate to a new version: the constraint is fundamental (a real failure class, not a one-off), it is inexpressible in current fields, and it recurs across independent systems. That conservatism is not inertia; it is what keeps the IR a stable contract while the knowledgebase around it compounds. The schema evolves on the timescale of fundamental discoveries; the heuristics evolve on the timescale of every failure the system is shown.

9 The Honest Limits

Five, plainly. **First**, the SIM's accuracy is the load-bearing assumption, and it is not yet proven — the blind cold-test described above is the experiment that will confirm or falsify it, and we regard that experiment as the single most important next step. **Second**, the gate's heuristics bite hardest on systems where failure classes live — concurrency, messaging, distributed state, control — and are correctly near-silent on simple CRUD; this is a tool for code that can hurt you, not for every line. **Third**, the evidence today is one reference run — an existence proof, not a measurement (Section 4.3).

Fourth, false alarms are an existential risk, not a nuisance. A deterministic gate that cries wolf will be disabled within a week — said by every reviewer of this paper, and true. The design answer is a severity *budget*: *blocking* findings must be rare and almost always real (if a team routinely overrides blocking findings, the gate — not the team — is wrong); *high* findings must be questions worth answering; the recall is carried by *medium* and *low*, which inform without stopping anything. Overrides are recorded with rationales in the decision memory, which makes false-alarm rate measurable per project rather than anecdotal — and makes the gate tunable without being amendable by the agent. The cold-test measures the gate's precision alongside code outcomes, because a gate nobody leaves enabled catches nothing.

Fifth-and-a-half, the third gate inherits the strengths *and* the blind spots of its underlying analyzers: a SAST tool that does not know a vulnerability class will not find it, and findings outside declared code locations route coarsely. The adapter mitigates (severity routing, tool versions recorded, multiple engines per ecosystem where they exist) but does not abolish this — code-level assurance is only as good as the ecosystem's tooling, which is precisely why the two gates above it had to exist.

Sixth, the vocabulary is a real cost. SIM, FAM, gate, heuristics, decision memory — four-and-a-bit coined terms for one workflow is a lot, and early readers said so, one of them in exactly these words: *it gives me a headache*. The terms are kept because each does distinct work — and because the alternative, prose, is what got us here — but the plain-language box at the top is not a courtesy; it is the admission price. A methodology that cannot explain itself without jargon does not survive contact with a team.

10 Two Claims, Kept Separate

It is worth being precise about what is being claimed, because there are two, and they have different burdens of proof.

The engineering contribution is a practical architecture: two intermediate representations between requirements and code — an intent-level one and a design-level one — a deterministic gate over each, an adapter that brings existing static analyzers to bear on the emitted code as a third gate, and a decision memory that records decisions. None of the individual parts is new; what we have not seen elsewhere is these parts assembled into this specific workflow. As engineering, this claim is *acceptable now* — it stands or falls on whether the assembly is coherent and buildable, which it demonstrably is.

The scientific hypothesis is stronger and not yet proven: that *forcing a coding agent to declare its intent, and gating that declaration deterministically, measurably improves the quality of the resulting software.* This is an empirical claim, and it is exactly what the validation protocol is designed to test. We state it as a hypothesis, not a result.

Keeping the two separate is not hedging; it is what makes the paper hard to attack. The architecture does not need the hypothesis to be true to be worth building, and the hypothesis does not need the architecture to be new to be worth testing.

11 Conclusion

The bottleneck in AI-assisted engineering is no longer the ability to generate code; it is the absence of any declared, checkable statement of what the code is meant to be. The tooling asymmetry is the tell: static analyzers for the emitted text exist in every ecosystem, while the two levels above the text — intent and design — had, until now, no equivalent at all, for humans or for machines. The System Intent Model addresses the upper half of that gap: an

intent-level intermediate representation, honest about absence and ignorance, written by the agent before it writes anything else. The Formal Architecture Model addresses the lower half: a design-level representation of the same system, total where the first is sparse, checked against it mechanically. Reviewing it is necessary because the failures that matter are failures of intent, best caught where intent is declared. Reviewing it deterministically is necessary because a stochastic judge cannot be a fixed rulebook. And requiring the agent to write it is feasible — provided we are honest enough to measure whether the declaration matches the deed. The agent proposes; the gates dispose — at the intent level, the design level, and the code level; and the decision memory remembers. It is, at minimum, a concrete and testable architecture for bringing declared intent into AI software generation — and if the hypothesis behind it holds, it suggests something broader: that AI programming, like compilation before it, may need explicit, staged intermediate representations to be trustworthy — not one, but one per level of commitment.

Appendix A The Questions, in Full

The claim that the gate is deterministic and auditable is exactly the claim that requires showing the list — so here it is. These are the questions the reviewer asks of any SIM, in natural language. (The canonical form is the implementation; the severities, applicability conditions, and per-domain tuning live in a curated knowledgebase outside this paper.)

#	The question
H1	What happens on each failure mode this architecture can actually have — component crash, message loss, dependency failure, restart loop? (Which modes apply depends on the declared architecture shape: a synchronous system cannot “lose” a message.)
H2	Every guarantee has a concrete mechanism — not an adjective? (“Fault tolerant” is not a mechanism; “restart, max 10 per minute, then escalate” is.)
H3	Is delivery coherent? May this important command be lost silently (at-most-once, no acknowledgement)? And where delivery is at-least-once, are the receivers idempotent under duplicates?
H4	Is recovery bounded — what stops a crash-looping component?
H5	No state with *pending work* waits forever on an external trigger with no timeout? (Waiting idle for the *next* message — a blocking dequeue — is sound: the thread is descheduled, nothing is in flight, and the runtime’s lifecycle bounds the wait. Waiting for *progress of work already started* with no deadline is not: that is the silent-hang failure class.)
H5e	If an actor's behavior cannot be enumerated (a learned policy, a genetic search) or is not reproducible (stochastic) — what deterministic envelope bounds it: valid input domain, guardrails, and fallback? The stochastic part proposes; the guard disposes.
H5f	Are continuous or stochastic outputs (torque, position size, valve %) bounded in range?
H5g	Whichever mailbox structure was chosen — are its failure modes addressed? Can a busy mailbox starve a low-priority one? Is lowest-priority dequeue latency bounded? Is the choice itself defended with a rationale? Are multiple output channels coherent?
H6	Is identity tied to a unique incarnation — not a reusable address or name — and are decisions made on a fresh view?
H7	Are safety and liveness guarantees verified beyond unit tests — fault injection, monitored invariants, model checking?

H8	Is each safety invariant enforced independently of the party it constrains — at the resource or platform layer, not by the same software it protects?
H9	Is every resource created per request or message bounded — a pool, a cap, an owner?
H10	Is any irreversible operation applied before validation, with no rollback — and is the rollback path itself reliable?
H11	Do two enforced invariants conflict under a mitigation operation (drain, failover, scale-down)? *Currently a manual question — the reviewer must answer it unaided.*
H12	Is a strong guarantee conditional on a precondition the runtime cannot enforce (user-code determinism, bounded clock skew) — and is that condition surfaced to operators?
H13	Can the system be stopped on command — enforced, inherited by child components, and not through a single point of failure?

H5e–H5g are property-based additions from the live corpus: the gate asks them not because an actor carries a particular label ("neural", "priority queue") but because its declared properties — enumerable, reproducible, continuous, prioritized — demand a matching verification standard. H14 (supply chain) arrived with schema v0.9: every external dependency declared, pinned, purposed — with the SBOM emitted as a byproduct.

The Second Gate, in Full (F1–F6)

#	The question (asked of the machine-readable FAM)
F1	Is every table complete — a row for every (state, event) pair, explicit unexpected-event rows, timeouts from waiting states, no orphan states? (Computed, not asked.)
F2	Does the behavior block match the declared properties — enumerable actors carry tables; non-enumerable ones carry complete envelopes (domain, guardrails, fallback, witnesses)?
F3	Is the FAM a lowering of the SIM — every SIM actor refined, no actor invented at blueprint level, traceability citations present?
F4	Do the FAM's ports match the SIM's interface — and does every continuous port carry a range/saturation bound?
F5	Is every guardrail, fallback, and invariant attached to a named code location? (An envelope without a location is a wish.)
F6	Is the code mirror ready — a handler named for every incoming message?

Gate 3 (code level) is an adapter over existing SAST tooling rather than a heuristic family of its own: findings are normalized into one schema, mapped onto FAM code locations, and routed by architectural role — blocking for high-confidence security/correctness findings in declared safety/enforcement paths. Examples of the mature tooling it adapts: VI Analyzer and JKI's J-Crawler (LabVIEW), Bandit/Semgrep/ESLint (text ecosystems), go vet/Clippy. The notable fact is not that code-level checking exists — it is that until this pipeline, *intent-level and design-level checking, for humans or machines, effectively did not.*